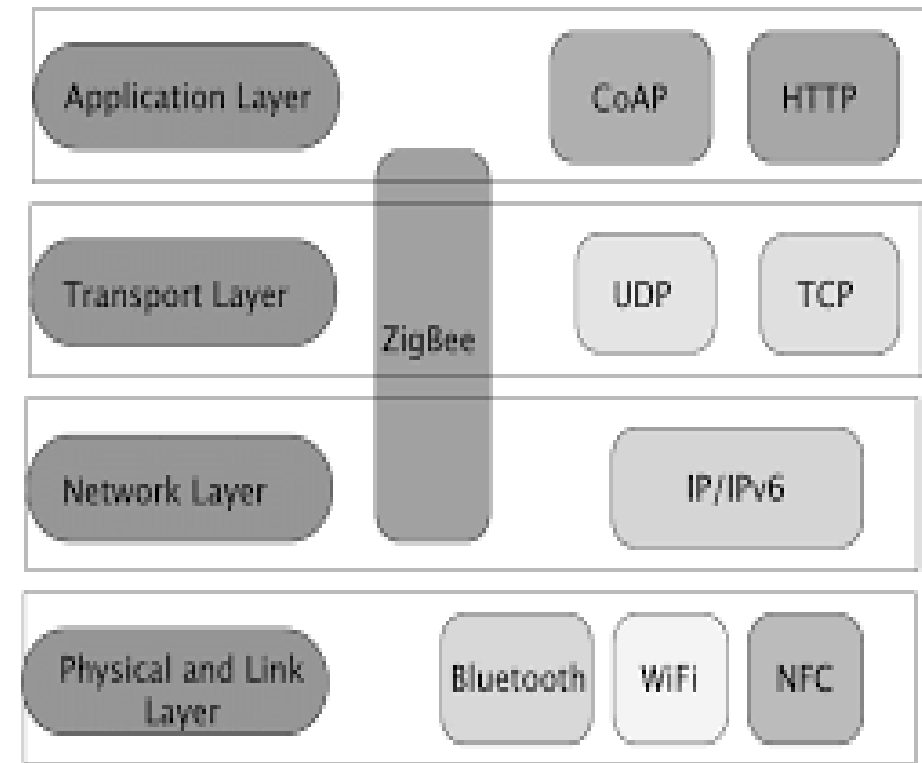
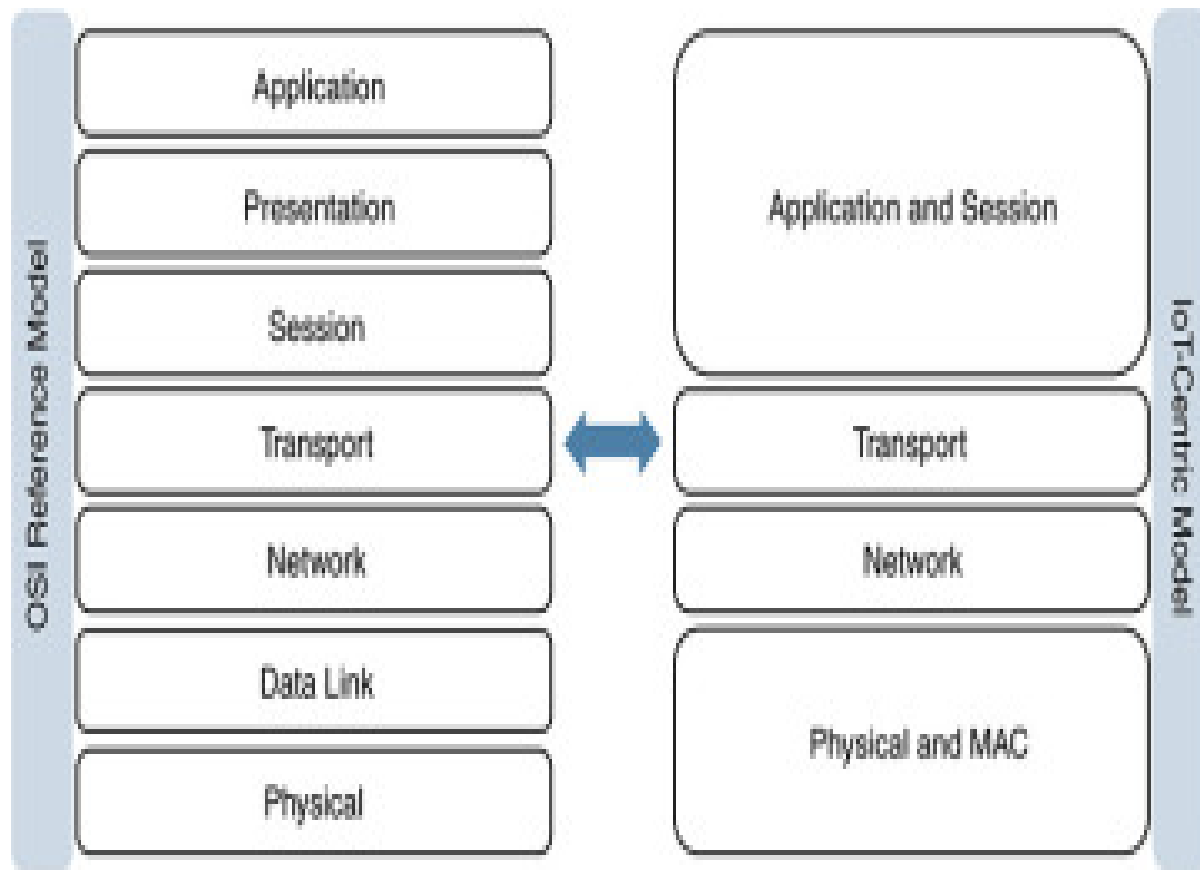


Introduction to Internet of Things

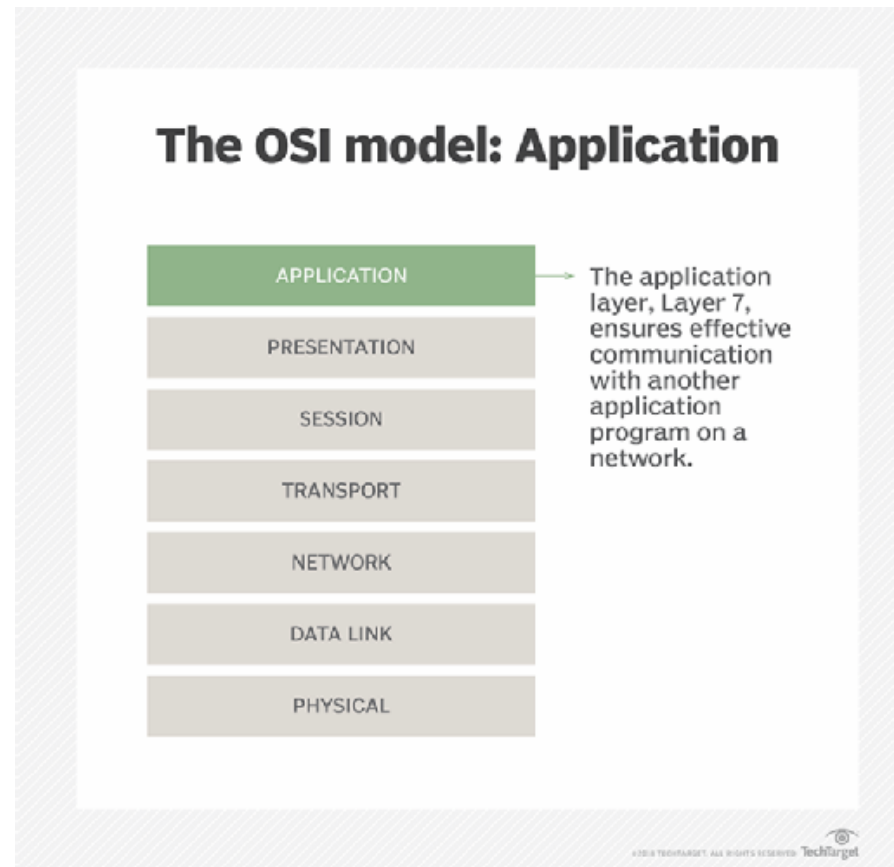
Lecture: 13 & 14

Communication Layers in IoT



Application Layer in IoT

- ▶ Applications are the programs that run on different end nodes.
- ▶ The application layer is the interface between the IoT device and the network with which it will communicate.



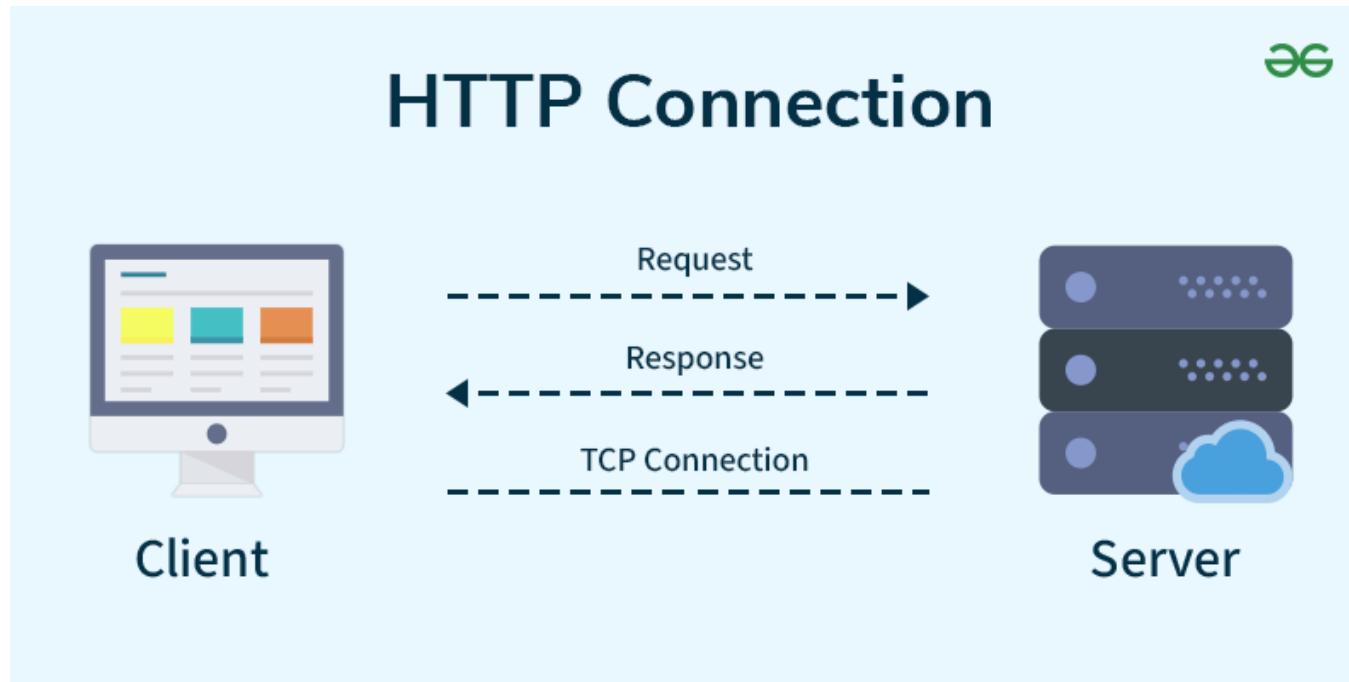
Application Layer Protocols

Important protocols used in application layer of IoT systems are:

- ▶ **Constrained Application Protocol (CoAP)**
- ▶ **Message Queue Telemetry Transport (MQTT)**
- ▶ **Extensible Message and Presence Protocol (XMPP)**
- ▶ **Advanced Message Queuing Protocol (AMQP)**
- ▶ **HyperText Transfer Protocol (HTTP)**
- ▶ **WebSocket Protocol**

HTTP/HTTPs

- ▶ It is the most popular application protocol used in the Internet or web.
- ▶ It is a request-response client-server protocol.
- ▶ It is state-less.
- ▶ It always runs over TCP/IP.



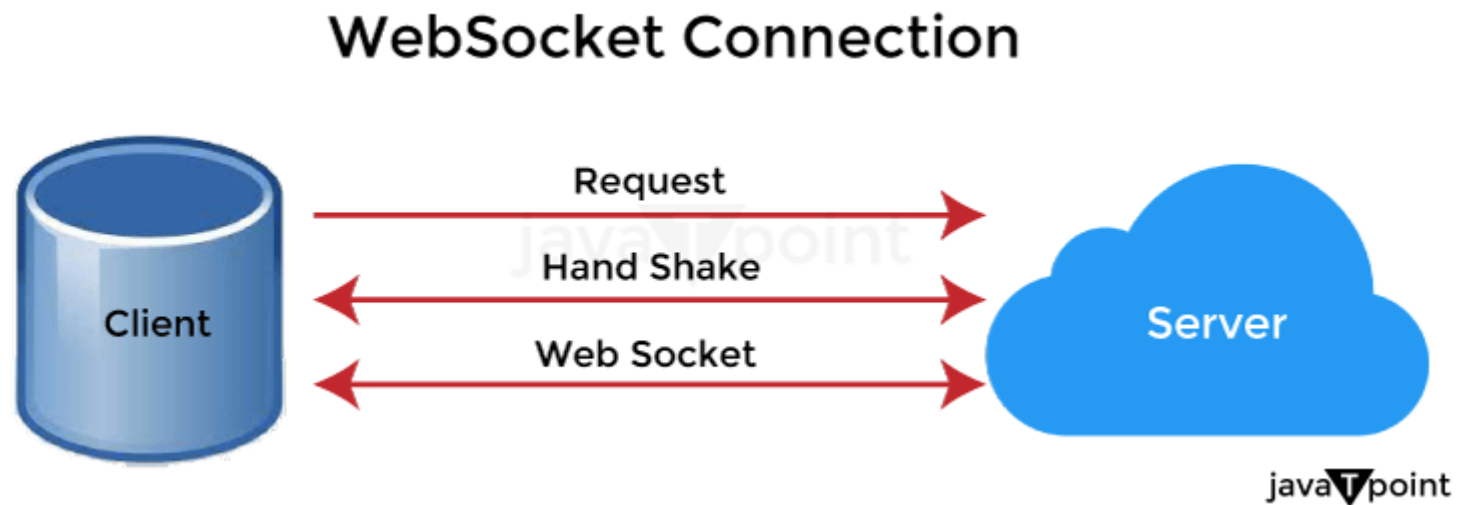
How HTTP works in IoT

- ▶ REST- Representational State Transfer.
- ▶ If any architecture in the web follows REST standards it is said to be RESTful.



WebSocket

- ▶ WebSocket is a communication protocol, providing bidirectional, full-duplex communication channels over a single TCP/IP connection between the client and the server.



CoAP (Constrained Application Protocol)

- ▶ It is a web transfer protocol that is used in constrained nodes or networks such as WSN, IoT, M2M etc.
- ▶ It is also known as Web of Things Protocol.
- ▶ It is an unreliable protocol due to the use of UDP.
- ▶ To make CoAP a reliable protocol, stop and wait with exponential backoff retransmission feature is incorporates in it. Duplicate detection is also introduced.

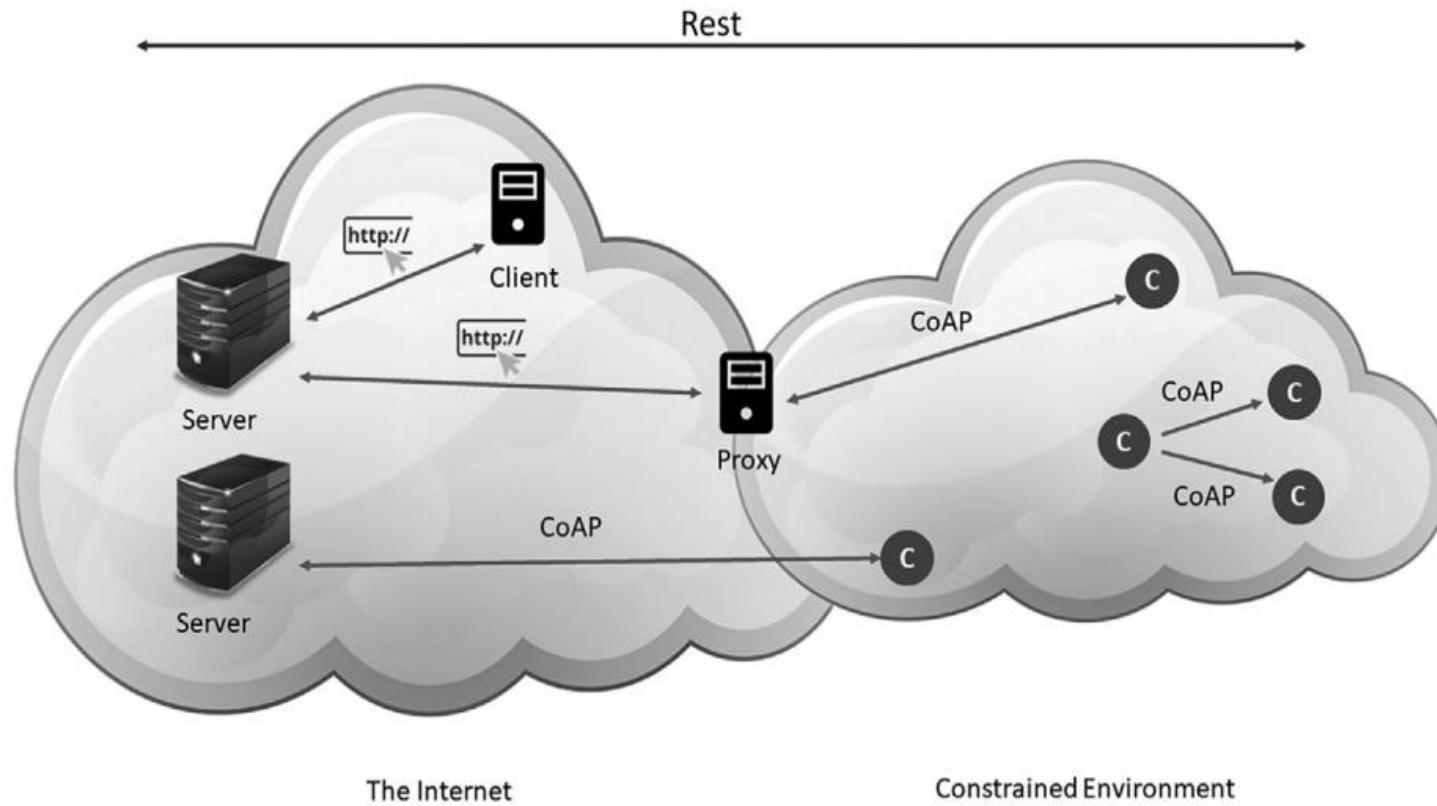
Features:

1. CoAP is more efficient in terms of bandwidth and energy consumption.
2. It follows a client-server (request/response) mechanism. Thus allow clients to perform CRUD operations on resources.
3. It supports lightweight publish/subscribe model.
4. It includes a resource observation feature that allows client to subscribe to a resource and receive notifications whenever the resource changes.

CoAP

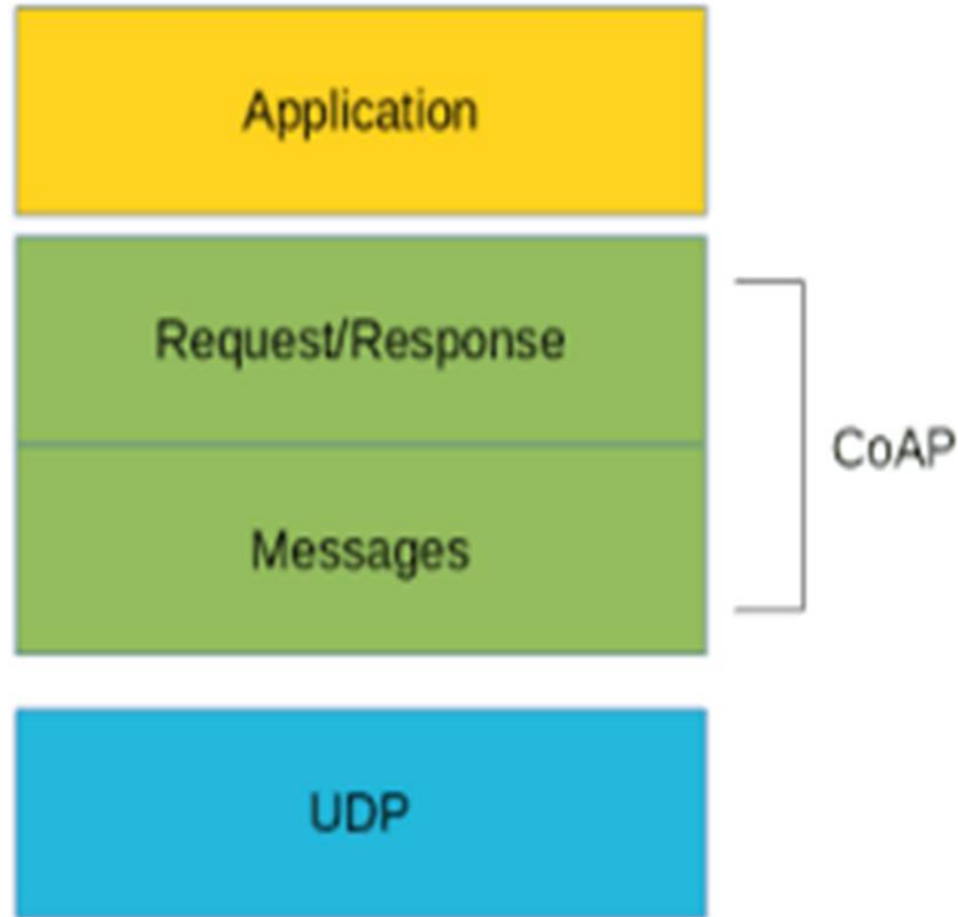
- ▶ CoAP's key features of lightweight transport, client-server model, support for resource observation, RESTful architecture, content negotiation, and resource discovery make it a suitable protocol for constrained devices and low-power networks, facilitating efficient communication and interaction within the IoT ecosystem.

CoAP Architecture



CoAP Layers

- ▶ Upper Layer is designed for communication method based on the Request-Response model.
- ▶ Lower Layer is designed to deal with UDP and asynchronous messages.



CoAP

- ▶ The devices in the network can act as a client or as a server.
- ▶ Each resource is identified by a URL, which allows clients to uniquely reference and interact with the resource.
- ▶ CoAP uses a request/response mechanism similar to HTTP.
- ▶ Requests are typically made using the methods GET, POST, PUT, and DELETE, which correspond to retrieving, creating, updating, and deleting resources, respectively.
- ▶ CoAP employs a lightweight message format based on the REST architectural style. Messages are exchanged using the UDP.
- ▶ CoAP message format consists of 4-byte header.

CoAP

- Four message types:
 - **Confirmable** – requires an ACK
 - **Non-confirmable** – no ACK needed
 - **Acknowledgement** – ACKs a Confirmable
 - **Reset** - indicates a Confirmable message has been received but context is missing for processing

CoAP message header

CoAP Message Header

Description

Ver	A 2-bit unsigned integer. It mentions the CoAP version number. Set to 1.
T	A 2-bit unsigned integer. Indicates message type, viz., confirmable (0), non-confirmable (1), ACK (2), or RESET (3).
TKL	A 4-bit unsigned integer, indicating the length of the token (0–8 bytes).
Code	An 8-bit unsigned integer, it is split into two parts, viz., 3-bit class (MSBs) and 5-bit detail (LSBs).
Message ID	A 16-bit unsigned integer, it is used for matching responses and to detect message duplication.

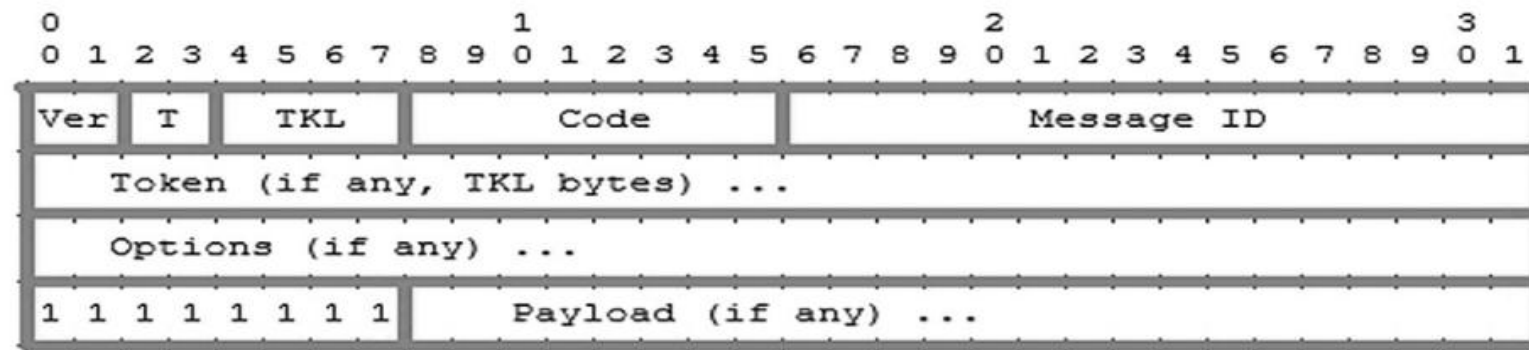
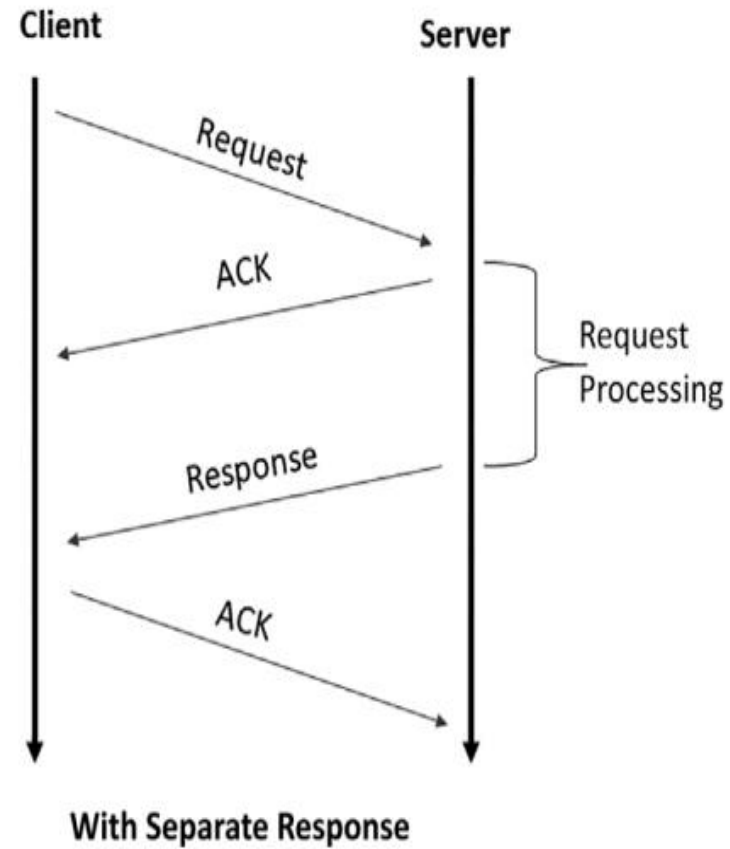
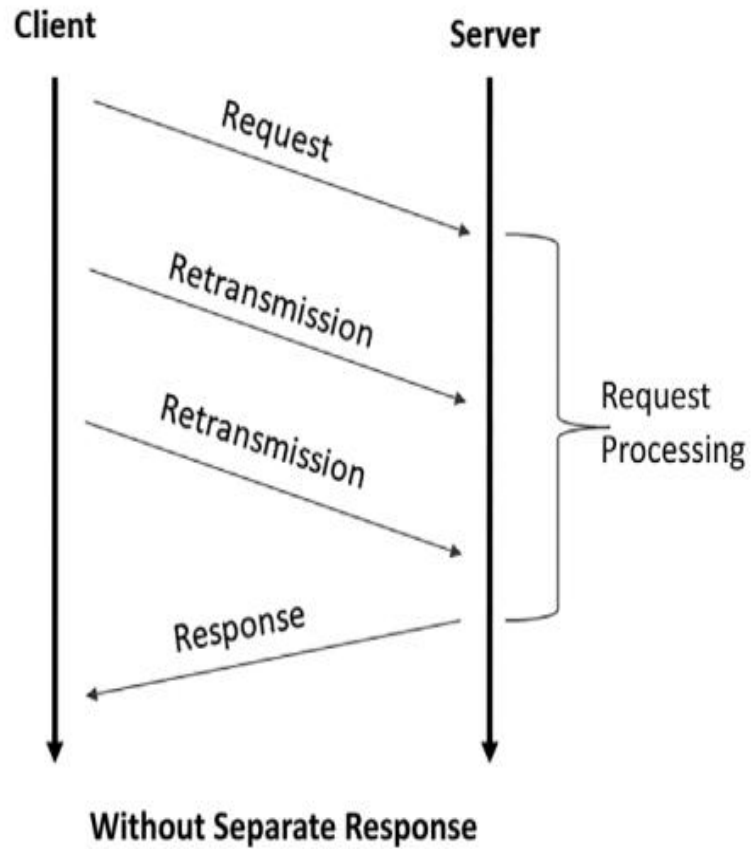


FIGURE 3.20 CoAP message format.

CoAP exchange messages



Steps for message exchange with CoAP

Step 1:Client Discovery: The client initiates the discovery process by sending a CoAP multicast message called a “UDP Discovery” to the CoAP multicast group address. This message allows the client to discover CoAP servers available on the network.

Step 2:Client Request: The client sends a CoAP request to a specific CoAP server. The request includes the following information.

- **Method:** Indicates the desired operation (e.g., GET, POST, PUT, DELETE).
- **URI (Uniform Resource Identifier):** Identifies the target resource on the server.
- **Message ID:** A unique identifier assigned by the client to match requests and responses.
- **Options:** Additional parameters like content type, observe, etc.
- **Payload:** Optional data to be sent along with the request.

Contd...

Step 3: Server Response: Contains following information:

- **Message ID:** Matches the request message ID for identification.
- **Code:** Indicates the response status (e.g., 205 Reset Content, 404 Not Found).
- **Options:** Additional parameters like content type, observe, etc.
- **Payload:** Optional data sent back to the client as a response.

Step 4: Confirmable (CON) message: CoAP messages can be either confirmable (CON) or non-confirmable (NON). A CON message expects an acknowledgment (ACK) from the recipient. If the client sends a CON message, it waits for an ACK before proceeding.

Step 5: Acknowledgment (ACK): When a CoAP server receives a CON message, it sends an ACK message back to the client to confirm message delivery. The ACK message contains the message ID of the received CON message.

Contd...

Step 6: Non-Confirmable (NON) Message: It does not require an acknowledgment. The server may choose to respond with a NON message if it does not require reliability.

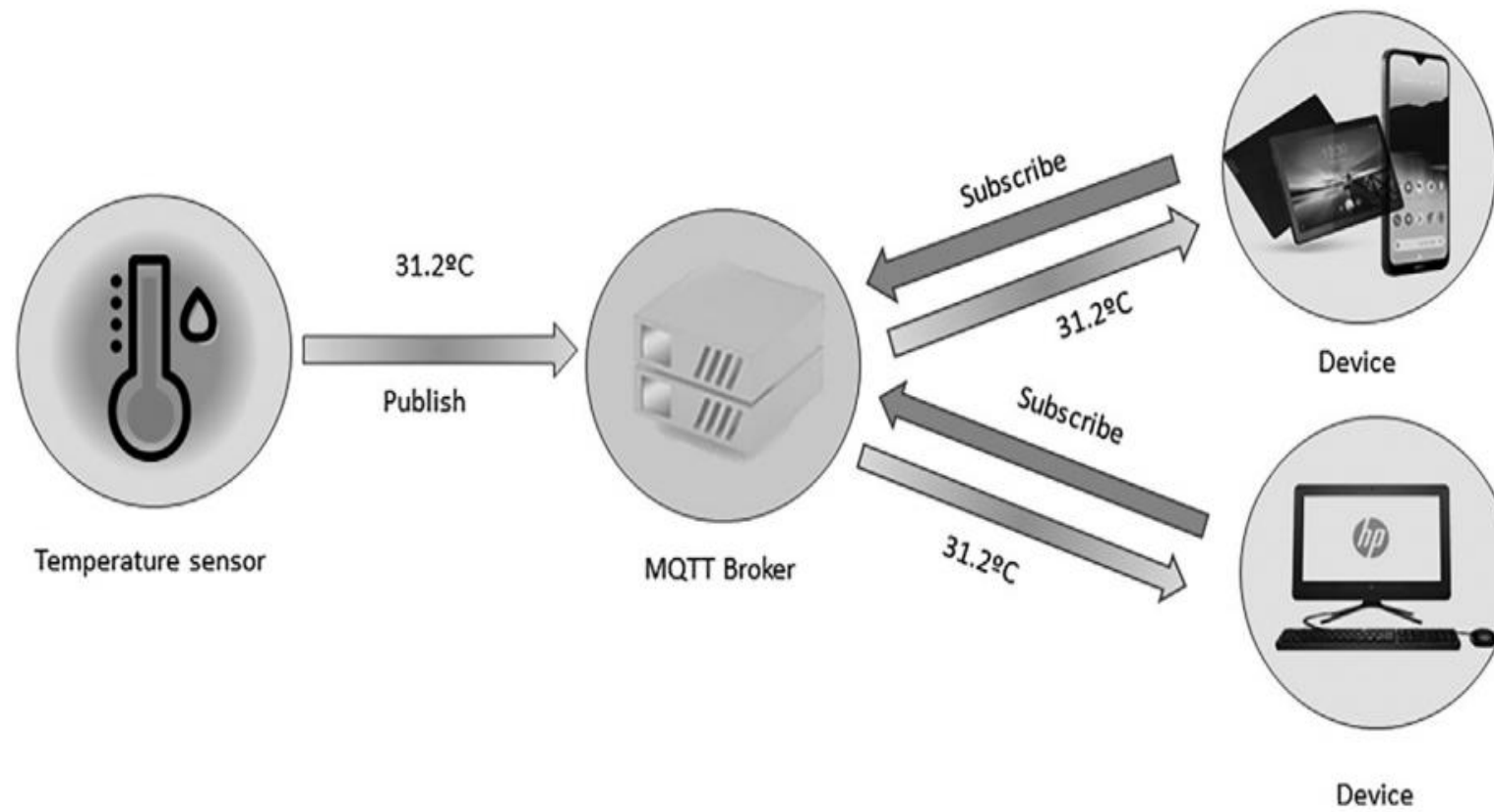
Step 7: Separate Response: In some cases, the server may decide to send the response in a separate CoAP message. This can be useful when the server needs more time to process the request or when the response is too large to fit in a single message. The client expects the separate response and waits for it.

Step 8: Message Retransmission: If the client does not receive an acknowledgment (ACK) for a CON message, it assumes the message was lost. The client retransmits the message. This ensures reliable delivery in the presence of unreliable networks.

MQTT (Message Queuing Telemetry Transport)

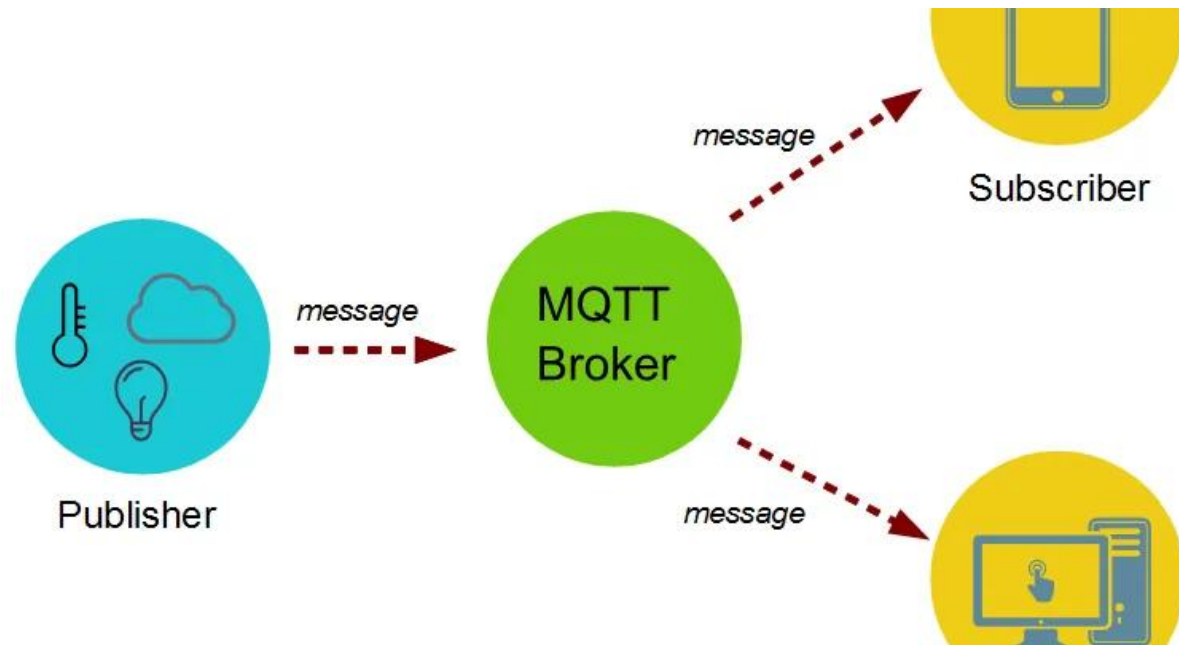
- ▶ It is a lightweight publish/subscribe messaging protocol.
- ▶ It is designed to provide reliable, efficient, and secure communication in resource constrained environments.
- ▶ It is a M2M connectivity protocol and is useful for connections with remote locations where network bandwidth is premium.
- ▶ It communicate with applications through the MQTT message broker.
- ▶ It runs over TCP protocol.
- ▶ MQTT can be used to connect large numbers of devices, making it an ideal choice for IoT applications that require scalability.

MQTT Architecture

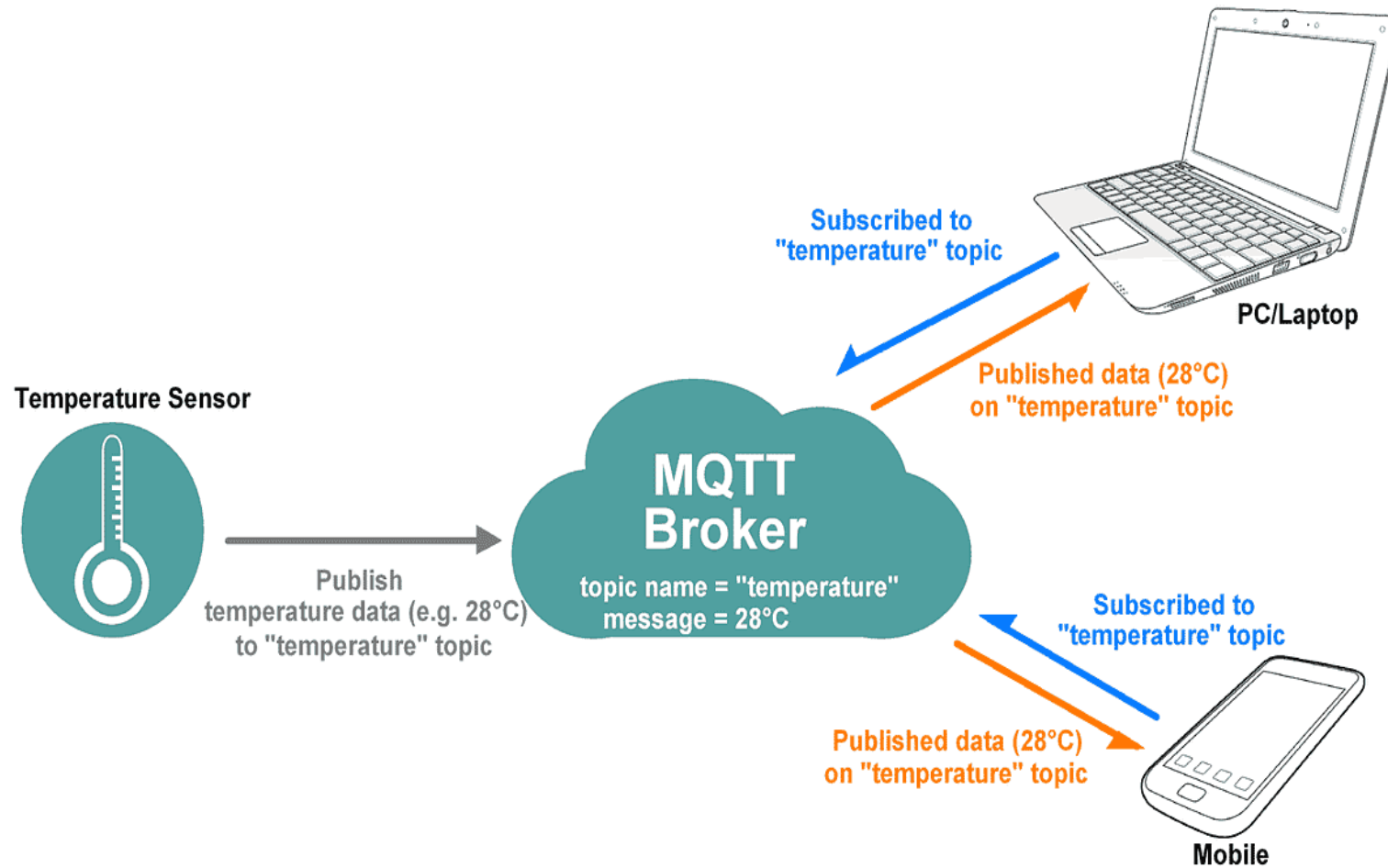


MQTT Components

1. Broker
2. Topic
3. Message
4. Client
 - Publisher
 - Subscriber



Working of MQTT



MQTT

- ▶ MQTT is not designed for real-time applications that require immediate response and high-speed data transfer.
- ▶ It is not designed for transferring large amounts of data, as it is optimized for lightweight messaging.
- ▶ The role of broker in IoT ecosystem is crucial, it can manage up to thousands of simultaneously connected MQTT clients.

Functions of MQTT Broker:

1. Accepting connections from publishers and subscribers.
2. Managing MQTT topics and subscriptions.
3. Filtering and routing messages between publishers and subscribers.
4. Managing the QoS levels of messages handling authentication and authorization of clients.
5. Providing logging and monitoring of message traffic.

MQTT Applications

- ▶ Industrial Automation
- ▶ Smart Home and Building Automation
- ▶ Medical and Healthcare
- ▶ Transportation
- ▶ Energy and Utilities
- ▶ Agriculture

MQTT vs CoAP

	MQTT	CoAP
Application Layer	Single Layered completely	Single Layered with 2 conceptual sub layers (Messages Layer and Request Response Layer)
Transport Layer	Runs on TCP	Runs on UDP
Reliability Mechanism	3 Quality of Services levels	Confirmable messages, Non-confirmable messages, Acknowledgments and retransmissions
Supported Architectures	Publish-Subscribe	Request-Response, Resource observe/Publish-Subscribe